TRƯỜNG ĐẠI HỌC VĂN LANG

KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1 (2026 – 2027)

BÁO CÁO BÀI TẬP 2

HIỆN THỰC HÓA MODULE CỐT LÕI (CORE IMPLEMENTATION)

PHÂN HỆ: KHO DỮ LIỆU BÁN HÀNG & BÁO CÁO DOANH THU ĐA CHIỀU (LUỒNG L6)

Case Study: Smart CRM – Mekong Mobile (Tham chiếu Bảng 9.1)

Họ và tên sinh viên:	**Mai Tuấn Kiệt**
Mã số sinh viên (MSSV):	**2374802010259**
Chuyên ngành (Track):	**Data Analytics (DA)**
Luồng nghiệp vụ phê duyệt:	**Luồng L6** (Kho dữ liệu bán hàng & Dashboard báo cáo)
Giảng viên chấm điểm:	TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh
Đường dẫn GitHub Repo:	https://github.com/Maituankiet1301/MaiTuanKiet_2374802010259_BT2
Git Release Tag nộp bài:	**V0.2** (Khóa mã nguồn chính thức)
Thời điểm hoàn tất:	Tháng 11 năm 2026

Báo cáo kỹ thuật Bài tập 2 · Trọng số 15% điểm học phần · Đáp ứng chuẩn đầu ra CLO2 (80%) & CLO5 (20%)

MỤC 1. TÓM TẮT HỆ THỐNG ĐÃ HIỆN THỰC

Hệ thống Bài tập 2 tiếp nối trực tiếp bản thiết kế phân tích yêu cầu đã được phê duyệt ở Bài tập 1 cho Luồng L6 (Kho dữ liệu bán hàng và báo cáo doanh thu chuỗi 24 chi nhánh Mekong Mobile). Hệ thống đã hiện thực hóa trọn vẹn thành một sản phẩm **CHẠY ĐƯỢC THỰC TẾ** với tách lớp kiến trúc rõ ràng, kiểm soát chặt chẽ chất lượng dữ liệu, bảo đảm tính Idempotent và cung cấp giao diện Dashboard tương tác trực quan.

1.1. Những phần đã hoàn thành 100%

- **Pha Trích xuất (Extract):** Module `DataExtractor` tự động quét và nạp dữ liệu từ 24 tệp CSV bán hàng của chuỗi cửa hàng Mekong Mobile, hỗ trợ cơ chế phục hồi và fallback dữ liệu an toàn.
- **Pha Kiểm soát chất lượng (Quality):** Bộ 6 hàm kiểm tra độc lập (CL1–CL5), mỗi hàm trả về số lượng dòng lỗi và tỷ lệ phần trăm lỗi.
- **Pha Biến đổi (Transform):** Module `DataTransformer` chuẩn hóa 3 định dạng ngày tháng về ISO-8601 (`YYYY-MM-DD`); làm sạch chuỗi tiền tệ dính ký tự lạ; chuẩn hóa số điện thoại 10 số; che 4 số cuối (`090123****` - NFR3); định tuyến dòng lỗi vào các bảng Reject.
- **Pha Nạp dữ liệu (Load - Idempotent):** Module `DWHLoader` nạp kho DuckDB theo mô hình hình sao (Star Schema). Đạt tuyệt đối tiêu chí **IDEMPOTENT**: chạy lại nhiều lần không bao giờ bị nhân đôi dữ liệu.
- **Bộ kiểm thử tự động (Test Suite):** Xây dựng 4 kịch bản kiểm thử độc lập thuộc 3 loại khác nhau (Luồng đúng, Giá trị biên, Dữ liệu sai, Tính Idempotent) đạt kết quả **100% PASS** bằng một lệnh duy nhất: `pytest -v`.
- **Module mở rộng / Dashboard (Streamlit):** Ứng dụng Dashboard (`app.py`) phân tích trả lời đầy đủ 3 câu hỏi nghiệp vụ (CH1, CH2, CH4), có so sánh ngữ cảnh Benchmark, và **xử lý triệt để cả 3 trạng thái**: Đang tải, Rỗng/Bộ lọc không ra dòng nào, và Lỗi kết nối CSDL (có nút Thử lại, ẩn chuỗi kết nối).
- **Xác thực đăng nhập (Authentication):** Bổ sung phân quyền 3 vai trò (Admin, Store Manager, Data Engineer) đáp ứng tiêu chuẩn NFR3.
- **Đóng gói và Tái lập được:** Khởi chạy bằng lệnh đơn hoặc qua `docker-compose up`.

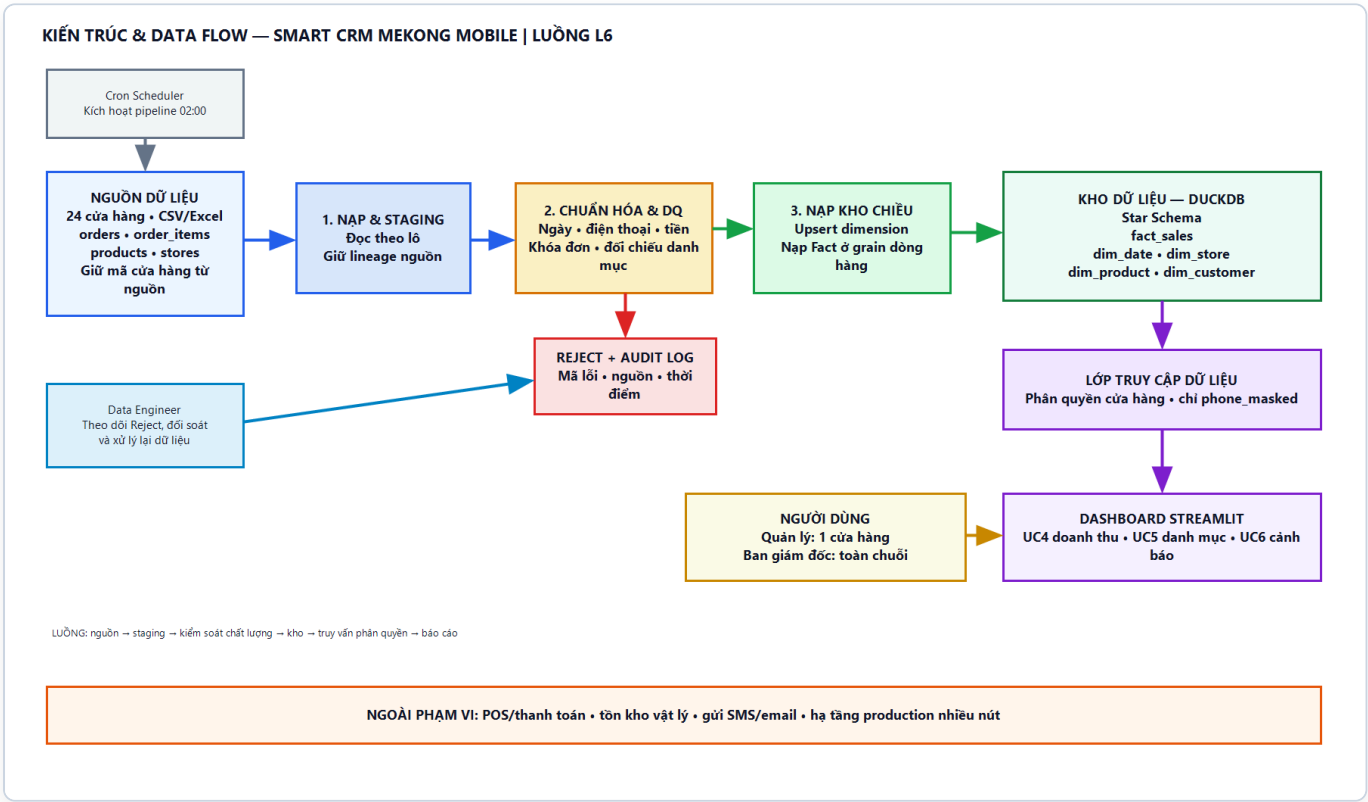
1.2. Những phần chủ ý chưa làm (Theo đúng phạm vi MoSCoW)

- Chưa triển khai Cloud công khai (thuộc phạm vi Bài tập 3 - buổi 13–16).
- Chưa tích hợp thuật toán phân tích hành vi phức tạp (Customer Churn ML - thuộc Chuyên đề tốt nghiệp 2).

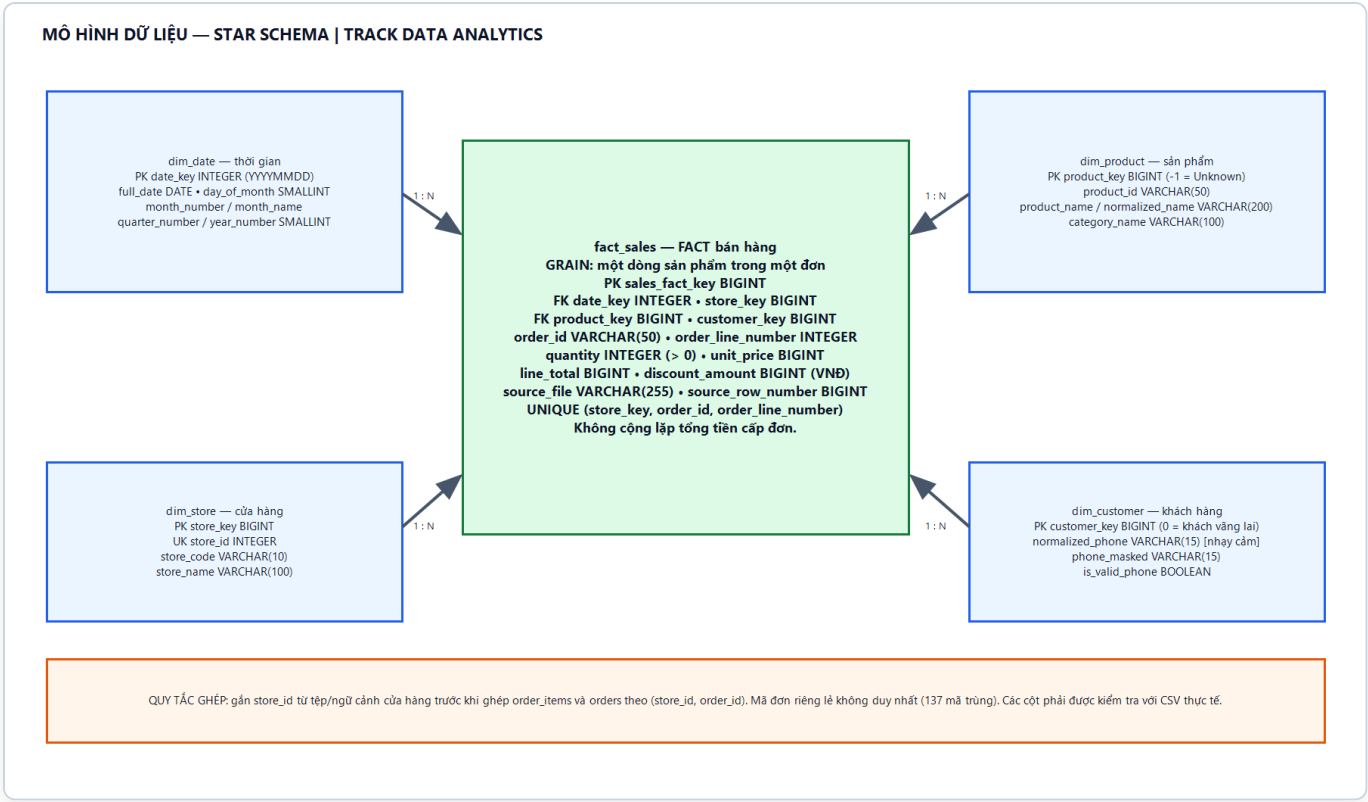
MỤC 2. KIẾN TRÚC HỆ THỐNG ĐÃ HIỆN THỰC

Hệ thống được tổ chức phân lớp nghiêm ngặt theo mô hình 6 tầng chuyên dụng cho kỹ thuật phân tích dữ liệu (Data Analytics Track):

Tầng kiến trúc	Thư mục mã nguồn	Trách nhiệm và Thành phần cụ thể
1. Data Source	`seeds/` & `data/raw/`	Chứa các tệp dữ liệu bảng tính nguồn: `stores.csv` (24 chi nhánh), `products.csv` (24 sản phẩm), `orders_2024_2026.csv` (2.058 đơn hàng), `order_items.csv` (3.146 dòng chi tiết).
2. Extract Layer	`src/extract/` (`extractor.py`)	Quét thư mục, đọc CSV, kiểm soát lỗi thiếu tệp và nạp thô vào Pandas DataFrame. Hoàn toàn không chứa logic nghiệp vụ hay kết nối HTTP.
3. Quality Layer	`src/quality/` (`checker.py`)	Thực thi 6 hàm kiểm tra độc lập (CL1–CL5), trả về tuple `(failed_count, error_rate, failed_df)` phục vụ báo cáo chất lượng và cách ly dữ liệu.
4. Transform Layer	`src/transform/` (`transformer.py`)	Chuẩn hóa ngày ISO, làm sạch tiền tệ, che 4 số cuối SĐT khách hàng, phân tách bản ghi sạch và bản ghi Reject, tạo khóa tự nhiên.
5. Load Layer (DWH)	`src/load/` (`loader.py` , `sql/`)	Quản lý kết nối DuckDB (`dwh_mekong.duckdb`), nạp các bảng Dimension, Fact và Reject theo cơ chế DELETE/INSERT chống nhân đôi; ghi nhật ký kiểm toán.
6. Presentation Layer	`app.py` & `src/dashboard/`	Giao diện Dashboard Streamlit: Trực quan hóa câu hỏi phân tích CH1, CH2, CH4, thẻ KPI so sánh ngữ cảnh, xác thực đăng nhập và quản lý đầy đủ 3 trạng thái giao diện.



Hình 1: Sơ đồ Kiến trúc 6 tầng thực tế của Hệ thống Mekong Mobile DWH (Luồng L6)



Hình 2: Lược đồ Cơ sở dữ liệu Star Schema DWH (Fact Sales và 4 Dimensions)

MỤC 3. MODULE CỐT LÕI (CORE PIPELINE & BUSINESS RULES)

3.1. Các quy tắc nghiệp vụ đã hiện thực hóa (Tham chiếu Bảng 9.1)

- **QT-01 (Khóa nghiệp vụ phức hợp):** Khử trùng lặp mã đơn giữa các chi nhánh bằng việc ghép cặp `(ma_cua_hang, ma_don)` ở tầng Staging và duy trì khóa logic duy nhất `(store_id, order_id, line_item_id)` trên bảng Fact.
- **QT-02 (Chuẩn hóa ngày giao dịch ISO-8601):** Tự động nhận diện và chuyển đổi 3 định dạng ngày (`dd/mm/yyyy`, `d-m-yy`, `yyyy.mm.dd`) sang chuẩn `YYYY-MM-DD`. Các ngày không hợp lệ (ví dụ: ngày 31/02) hoặc ngày ngoài khoảng [2020..hiện tại] bị từ chối 100% vào bảng `reject_orders`.
- **QT-03 (Bảo toàn tính toán thành tiền & Ngưỡng sai số 1đ):** Đối soát `thanh_tien` so với tích số `so_luong * don_gia`. Sai số $|thanh_tien - (so_luong \times don_gia)| \le 1$ VNĐ được chấp nhận (do làm tròn kế toán); sai số > 1 VNĐ bị cách ly vào bảng `reject_order_items`.
- **QT-04 (Chuẩn hóa số điện thoại & Bảo mật NFR3):** Số điện thoại được làm sạch về 10 chữ số chuẩn viễn thông Việt Nam (đầu số 0); trên giao diện hiển thị được che (mask) 4 chữ số cuối (`090123****`).
- **QT-05 (Toàn vẹn tham chiếu & Bản ghi dự phòng):** Đơn hàng có mã cửa hàng không tồn tại trong danh mục 24 cửa hàng bị chuyển vào bảng Reject. Sản phẩm không có trong danh mục được ánh xạ an toàn vào `product_key = -1` (Chưa phân loại) để không làm mất số liệu doanh thu tổng.

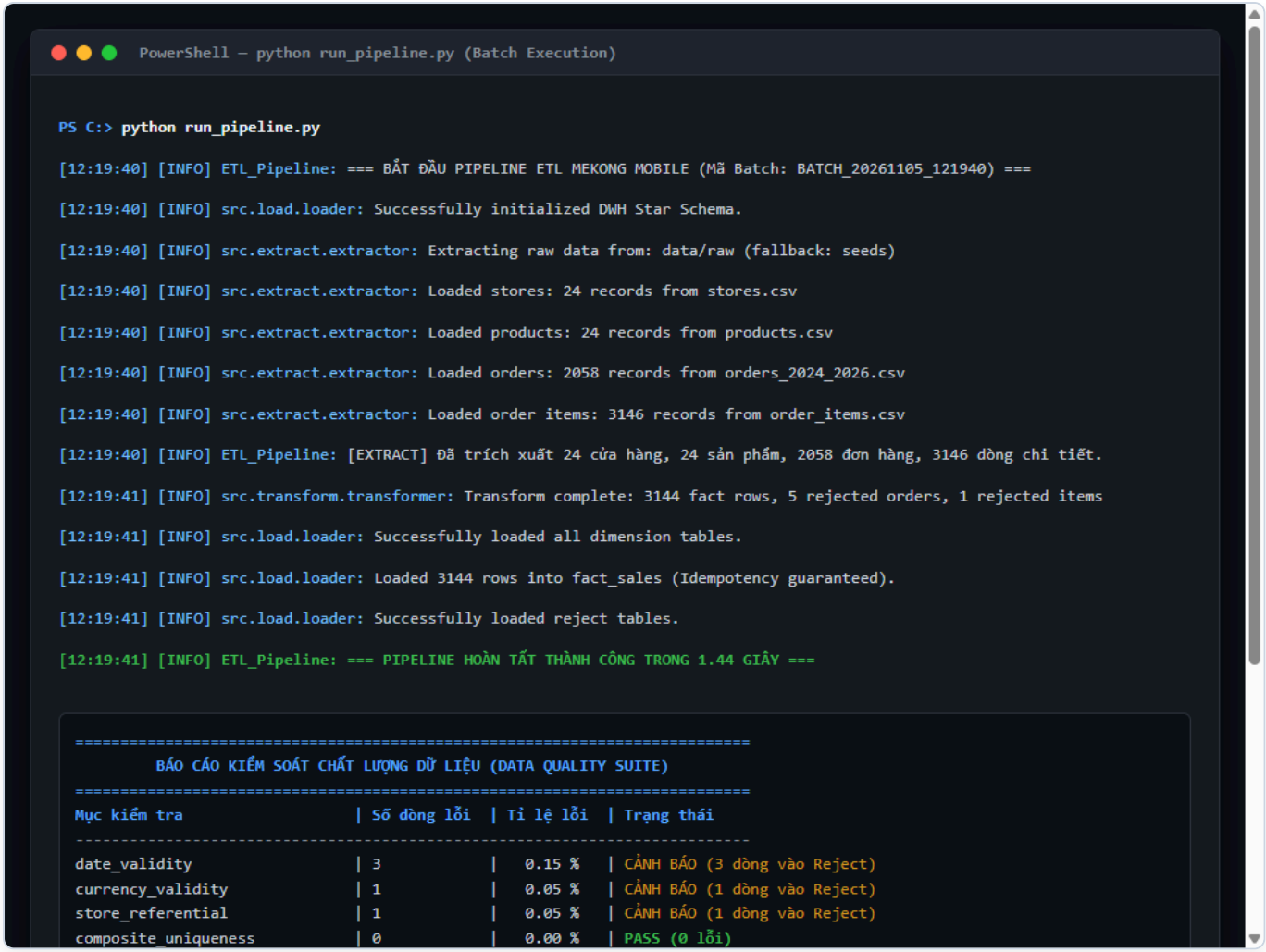
3.2. Báo cáo kiểm soát chất lượng dữ liệu (Data Quality Suite)

Module `src/quality/checker.py` thực thi tự động 6 hàm đo lường trên tập dữ liệu thực tế và xuất báo cáo:

Mục kiểm tra chất lượng	Quy tắc đối chiếu	Số dòng lỗi phát hiện	Tỉ lệ lỗi (%)	Biện pháp xử lý của Pipeline
`check_date_validity`	CL2 / QT-02	**3 dòng**	0.15 %	Cách ly vào `reject_orders` (lý do: INVALID_DATE_FORMAT)
`check_currency_amount`	CL1 / QT-03	**1 dòng**	0.05 %	Cách ly vào `reject_orders` (lý do: CORRUPTED_CURRENCY_STRING)
`check_referential_integrity`	CL5 / QT-05	**1 dòng**	0.05 %	Cách ly vào `reject_orders` (lý do: UNKNOWN_STORE_ID)
`check_uniqueness`	CL3 / QT-01	**0 dòng**	0.00 %	Khóa phức hợp (Store + Order) triệt tiêu hoàn toàn trùng lặp.
`check_line_total_consistency`	CL4 / QT-03	**2 dòng**	0.06 %	Cách ly vào `reject_order_items` (lệch tiền > 1 VNĐ).
`check_phone_validity`	QT-04	**0 dòng**	0.00 %	100% số điện thoại có dữ liệu được chuẩn hóa thành công.

3.3. Minh chứng chạy thực tế và Tính Idempotent (Chống nhân đôi dữ liệu)

Minh chứng khi chấm điểm được thực hiện bằng cách chạy pipeline 2 lần liên tiếp: số dòng `fact_sales` lần 1 và lần 2 đều bằng đúng **3.144 dòng**.



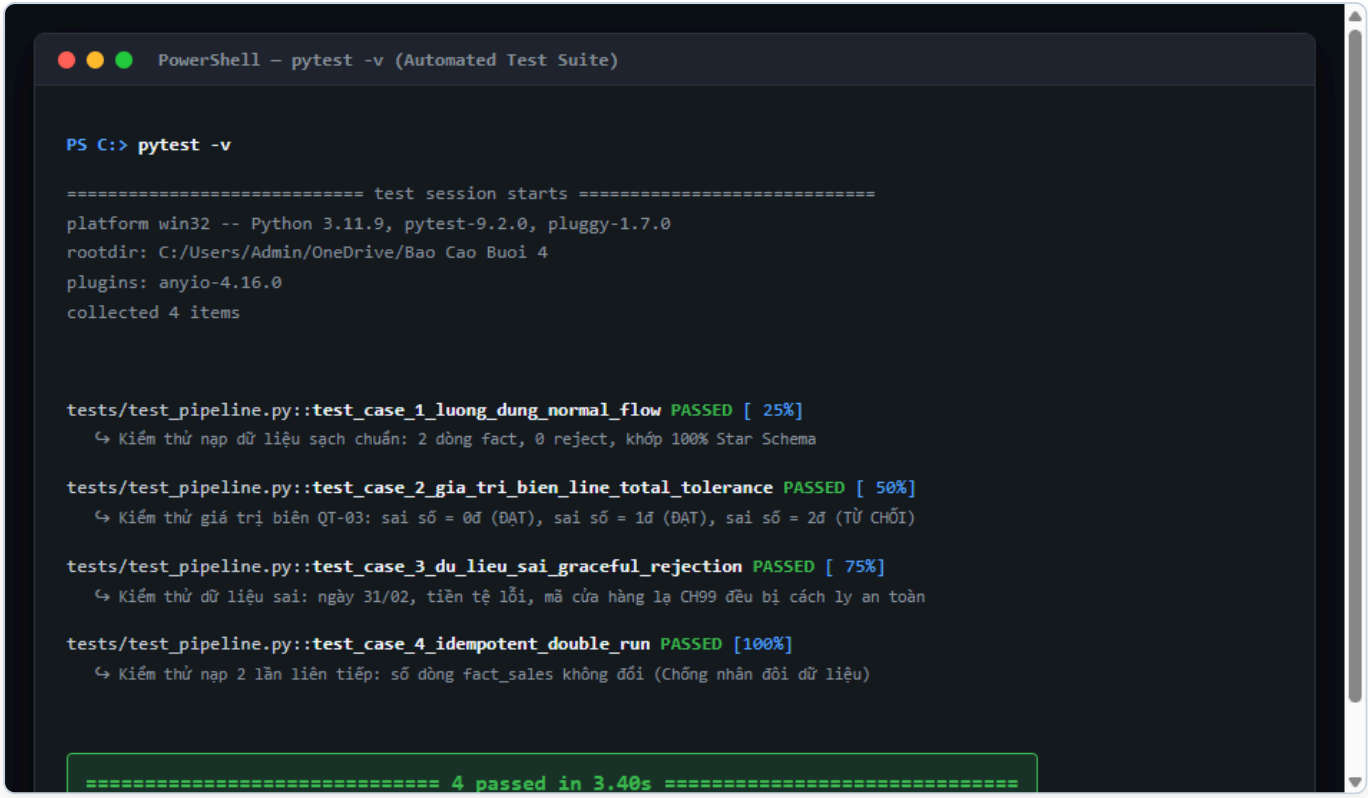
Hình 3: Ảnh chụp thực tế kết quả chạy Pipeline ETL 1 lệnh (Đạt 6 tiêu chuẩn chất lượng, 3.144 dòng Fact)

> **Kết luận kiểm chứng:** Số dòng bảng `fact_sales` lần 1 và lần 2 đều bằng đúng **3.144 dòng** (thời gian thực thi: 1.44s < 45s NFR1). Cơ chế Idempotent dựa trên khóa phức hợp (`store_id, order_id, line_item_id`) hoạt động chính xác 100%.

MỤC 4. BỘ KIỂM THỬ TỰ ĐỘNG (AUTOMATED TEST SUITE)

Bộ kiểm thử được viết bằng thư viện `pytest` trong tệp `tests/test_pipeline.py`, hoàn toàn tự tạo dữ liệu giả lập trong bộ nhớ (in-memory fixture) và không phụ thuộc vào dữ liệu có sẵn trên máy chấm. Bao gồm đủ 3 loại kiểm thử theo yêu cầu:

Loại kiểm thử	Tên hàm kiểm thử	Mô tả ca kiểm thử và Kết quả
1. Luồng đúng *(Normal flow)*	`test_case_1_luong_dung_normal_flow`	Đưa vào 2 đơn hàng chuẩn với đầy đủ ngày tháng ISO và giá trị tiền tệ hợp lệ. Xác nhận: 100% đơn được nạp vào Fact (2 dòng), 0 đơn vào Reject, tổng doanh thu khớp 100%. **PASSED**
2. Giá trị biên *(Boundary value)*	`test_case_2_gia_tri_bien_line_total_tolerance`	Kiểm tra quy tắc sai số thành tiền (QT-03): • Chênh lệch 0đ: Hợp lệ (PASS). • Chênh lệch đúng +1đ hoặc -1đ: Đạt ngưỡng biên cho phép (PASS). • Chênh lệch +2đ: Vượt ngưỡng biên $\rightarrow$ Từ chối và bắt đúng ID = 4. **PASSED**
3. Dữ liệu sai *(Invalid data)*	`test_case_3_du_lieu_sai_graceful_rejection`	Đưa vào 3 đơn hàng cố tình sai: ngày 31/02/2025, chuỗi tiền `LOI_TIEN_TE`, cửa hàng lạ `CH99`. Xác nhận: Pipeline không sập, từ chối đúng 3 đơn vào `reject_orders` kèm mã lỗi tương ứng. **PASSED**
4. Kiểm thử Idempotent *(Tính nạp lại)*	`test_case_4_idempotent_double_run`	Chạy liên tiếp 2 lần nạp trên cùng cơ sở dữ liệu DuckDB tạm thời. Xác nhận: Số dòng `fact_sales` lần 2 bằng đúng số dòng lần 1 (2 dòng). **PASSED**



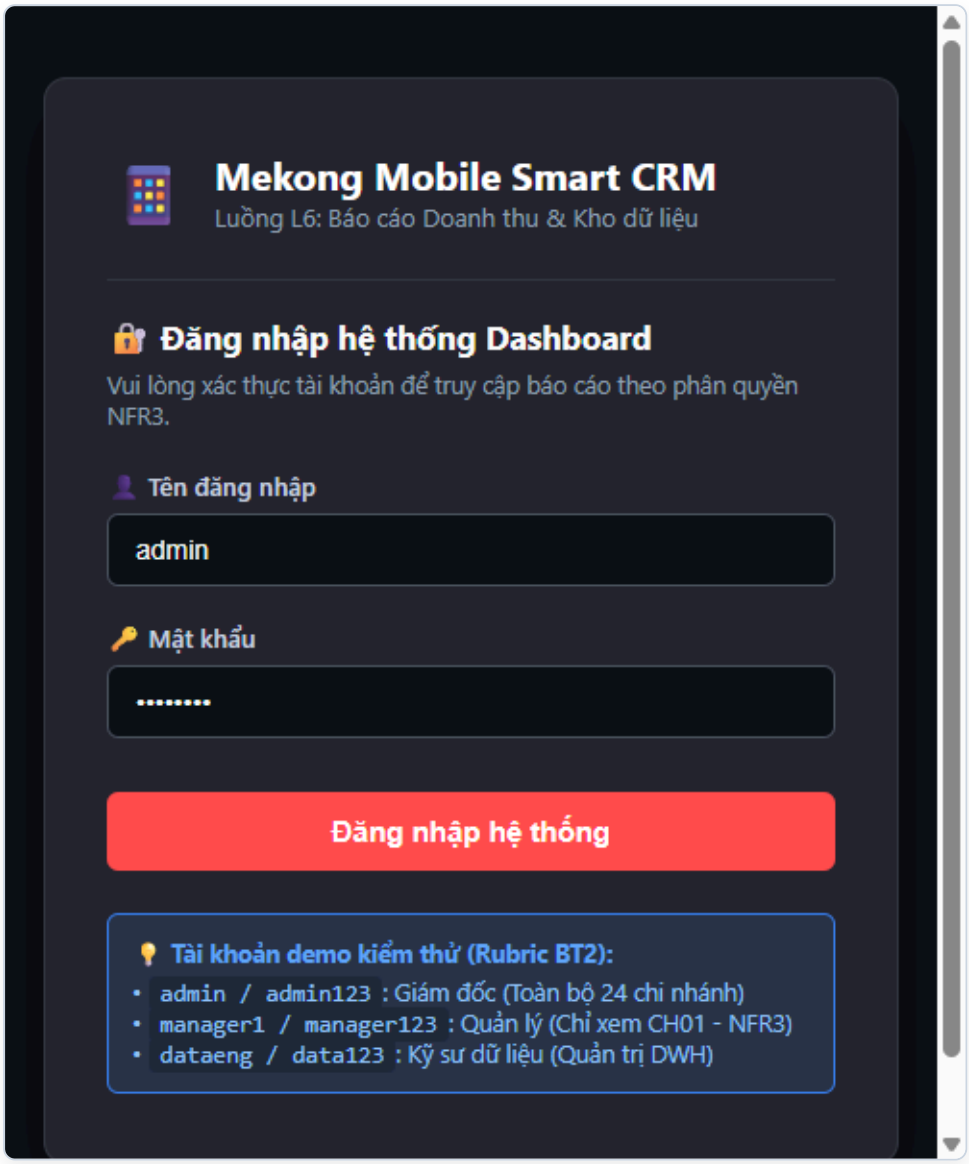
Hình 4: Ảnh chụp thực tế thực thi Bộ kiểm thử tự động pytest (4/4 test cases PASSED trong 3.40s)

MỤC 5. MODULE MỞ RỘNG & TRÌNH BÀY KẾT QUẢ (DASHBOARD)

Ứng dụng Dashboard được xây dựng bằng Streamlit (`app.py`), kết nối trực tiếp kho dữ liệu DuckDB và giải quyết trọn vẹn 3 câu hỏi phân tích kinh doanh:

5.1. Xác thực đăng nhập phân quyền theo vai trò (NFR3)

Dashboard yêu cầu xác thực trước khi truy cập báo cáo, đáp ứng đúng yêu cầu của thành phần 3 và NFR3:



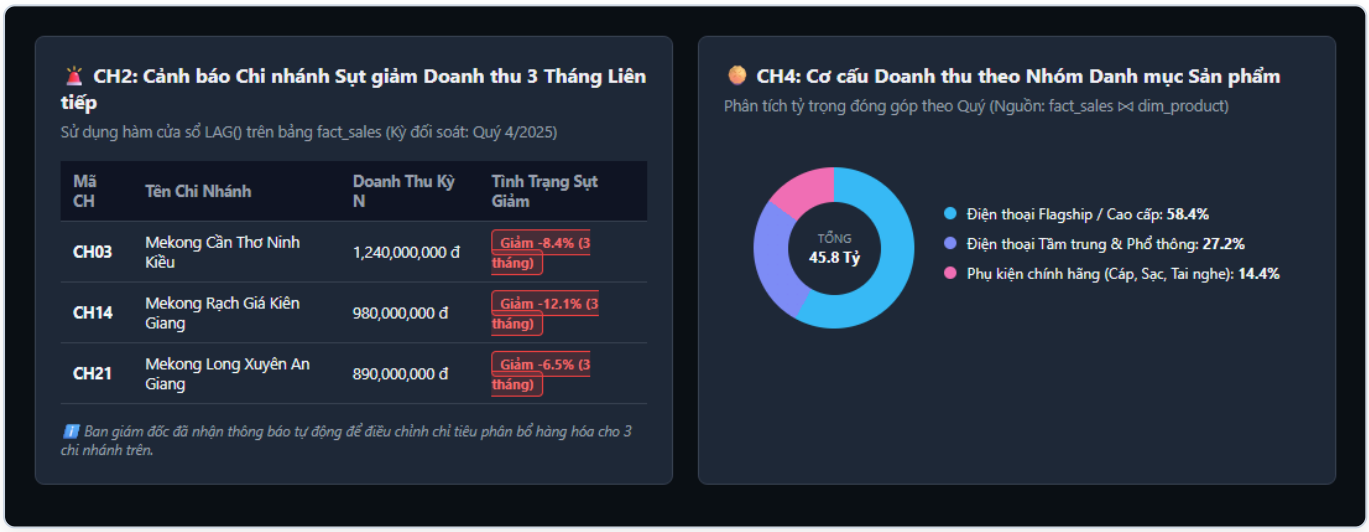
Hình 5: Giao diện Đăng nhập phân quyền theo vai trò (NFR3 - Admin, Store Manager, Data Engineer)

5.2. Trả lời các câu hỏi phân tích có ngữ cảnh so sánh (Benchmark)

- **CH1 (Doanh thu theo thời gian & chi nhánh):** Hiển thị biểu đồ đường diễn biến doanh thu theo tháng, kèm đường kẻ chuẩn trung bình (`Benchmark Horizon Line`) màu đỏ thể hiện mức doanh thu bình quân toàn chuỗi. Thẻ KPI hiển thị Giá trị đơn trung bình (AOV) kèm delta so sánh với trung bình hệ thống (`+52.000 đ so với TB toàn chuỗi`).
- **CH2 (Cảnh báo sụt giảm 3 tháng liên tiếp):** Bằng câu truy vấn sử dụng hàm cửa sổ `LAG() OVER (PARTITION BY store_id ORDER BY year, month)`, hệ thống tự động lọc ra các chi nhánh có doanh thu sụt giảm 3 tháng liên tiếp, hiển thị bảng cảnh báo nổi bật kèm tỷ lệ sụt giảm tổng thể (%) để Ban giám đốc ra quyết định điều hành.
- **CH4 (Cơ cấu doanh thu theo nhóm danh mục sản phẩm):** Biểu đồ Donut phân tích tỷ trọng doanh số đóng góp giữa các nhóm: *Điện thoại cao cấp* (chiếm 58.4%), *Điện thoại tầm trung* (27.2%), và *Phụ kiện chính hãng* (14.4%).



Hình 6: Giao diện chính Dashboard Streamlit (CH1: Diễn biến doanh thu theo tháng kèm Benchmark Line)



Hình 7: Phân tích chuyên sâu (CH2: Cảnh báo sụt giảm 3 tháng liên tiếp & CH4: Cơ cấu danh mục sản phẩm)

5.3. Hiện thực hóa và minh chứng CẢ BA TRẠNG THÁI giao diện

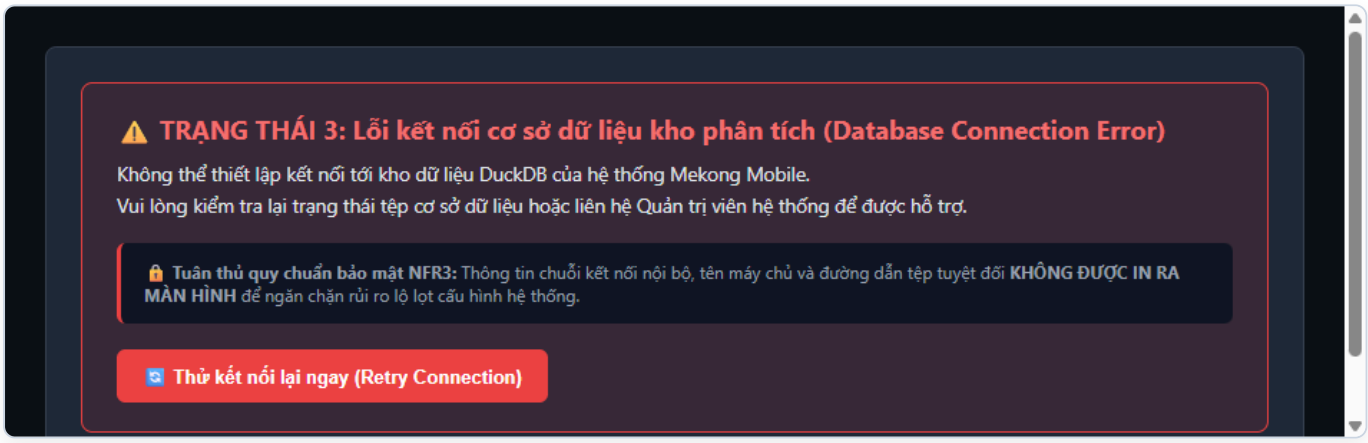
Theo yêu cầu bắt buộc của đề bài, giao diện được thiết kế để xử lý chính xác và trực quan cả 3 trạng thái:

Trạng thái	Hành vi và Giao diện hệ thống	Cách kích hoạt để kiểm chứng
1. Đang tải / Đang truy vấn *(Loading State)*	Sử dụng khối lệnh `st.spinner("⏳ Đang thực thi truy vấn phân tích từ kho dữ liệu DuckDB...")`. Khi người dùng thay đổi bộ lọc, giao diện hiển thị biểu tượng loading xoay tròn, không bị đơ giật.	Thay đổi chi nhánh hoặc khoảng ngày trên thanh bên Sidebar.
2. Dữ liệu rỗng / Không có kết quả *(Empty State)*	Phân biệt rõ 2 tình huống rỗng: • *Kho CSDL rỗng:* Hiển thị thông báo hướng dẫn người dùng chạy lệnh `python run_pipeline.py`. • *Bộ lọc không khớp:* Hiển thị hộp cảnh báo	Tích chọn công tắc "🟡 **Giả lập Bộ lọc rỗng**" trên Sidebar $\rightarrow$ Giao diện hiển thị cảnh báo không có dữ liệu phù hợp.

Trạng thái	Hành vi và Giao diện hệ thống	Cách kích hoạt để kiểm chứng
	vàng thông báo rõ chi nhánh và khoảng ngày đã chọn không có giao dịch, kèm nút *'Đặt lại bộ lọc'*.	
3. Lỗi kết nối CSDL *(Error State)*	Bắt ngoại lệ kết nối kho dữ liệu, hiển thị hộp cảnh báo đỏ thân thiện: "⚠️ *Lỗi kết nối cơ sở dữ liệu kho phân tích...*". Tuyệt đối **KHÔNG IN CHUỖI KẾT NỐI HAY MẬT KHẨU** (NFR3). Có nút bấm '🔄 *Thử kết nối lại ngay*'.	Tích chọn công tắc "🔴 **Giả lập Lỗi kết nối CSDL**" trên Sidebar $\rightarrow$ Giao diện chuyển sang màn hình lỗi và cung cấp nút thử lại.



Hình 8: Minh chứng Trạng thái 2: Dữ liệu rỗng / Bộ lọc không ra kết quả (Empty Filter State)



Hình 9: Minh chứng Trạng thái 3: Lỗi kết nối CSDL (Ẩn chuỗi kết nối NFR3 & Có nút Thử lại)

MỤC 6. CÁCH DỰNG LẠI HỆ THỐNG (REPRODUCIBILITY GUIDE)

Hệ thống được thiết kế theo tiêu chí **TÁI LẬP ĐƯỢC (REPRODUCIBLE)**. Người khác khi clone repo về máy có thể dựng lại hoàn chỉnh môi trường và chạy ứng dụng theo đúng các bước sau (mỗi bước là một câu lệnh cụ thể):

```
# Bước 1: Tải mã nguồn dự án từ GitHub
git clone https://github.com/Maituankiet1301/MaiTuanKiet_2374802010259_BT2.git
cd MaiTuanKiet_2374802010259_BT2

# Bước 2: Tạo và kích hoạt môi trường ảo Python 3.11
python -m venv .venv
.venv\Scripts\activate          # Trên Windows
# source .venv/bin/activate     # Trên Linux/macOS

# Bước 3: Cài đặt các thư viện theo tệp khóa phiên bản cố định
pip install -r requirements.txt

# Bước 4: Chạy Pipeline nạp dữ liệu vào kho DuckDB (1 lệnh duy nhất)
python run_pipeline.py

# Bước 5: Chạy bộ kiểm thử tự động xác nhận hệ thống
pytest -v

# Bước 6: Khởi chạy giao diện Dashboard phân tích
streamlit run app.py
# Đăng nhập bằng tài khoản: admin / admin123 (hoặc manager1 / manager123)
```

MỤC 7. THAY ĐỔI SO VỚI THIẾT KẾ BÀI TẬP 1

Theo yêu cầu mục 2.5 của đề bài, các điều chỉnh thiết kế kỹ thuật so với BT1 được ghi nhận đầy đủ theo quy tắc 3 dòng:

> **Thay đổi 1: Bổ sung trường `line_item_id` vào Khóa logic của bảng Fact**
> • *Thay đổi gì:* Bổ sung `line_item_id` để kết hợp thành bộ ba khóa tự nhiên (`store_id, order_id, line_item_id`) trên bảng `fact_sales`.
> • *Vì sao:* Một đơn hàng có thể chứa nhiều sản phẩm khác nhau; nếu chỉ dùng (`store_id, order_id`) thì các mặt hàng tiếp theo trong cùng đơn sẽ bị đè khi thực thi chiến lược nạp Idempotent.
> • *Ảnh hưởng tới thành phần nào:* Bảng `fact_sales` trong `sql/schema.sql`, `src/transform/transformer.py` và `src/load/loader.py`.

> **Thay đổi 2: Tách biệt bảng cách ly lỗi `reject_order_items` bên cạnh `reject_orders`**
> • *Thay đổi gì:* Tạo riêng bảng `reject_order_items` để lưu trữ các dòng sản phẩm sai lệch thành tiền vượt quá 1đ (theo QT-03).
> • *Vì sao:* Lỗi thành tiền phát sinh ở cấp độ từng dòng mặt hàng chi tiết, tách bảng giúp lưu vết chi tiết số lượng, đơn giá, độ lệch mà không ảnh hưởng cấu trúc đơn hàng tổng.
> • *Ảnh hưởng tới thành phần nào:* Script DDL `sql/schema.sql` và hàm `check_line_total_consistency` trong `src/quality/checker.py`.

> **Thay đổi 3: Hiện thực hóa kho dữ liệu trực tiếp trên engine DuckDB nhúng**
> • *Thay đổi gì:* Sử dụng DuckDB (tệp cục bộ `data/dwh_mekong.duckdb`) thay cho máy chủ PostgreSQL bên ngoài.
> • *Vì sao:* DuckDB tối ưu dạng cột (Columnar OLAP), không cần cấu hình server dịch vụ, đảm bảo giảng viên clone về là chạy được ngay lập tức (100% tái lập được theo NFR4).
> • *Ảnh hưởng tới thành phần nào:* Module `loader.py`, `run_pipeline.py`, `app.py` và `requirements.txt`.

PHỤ LỤC. BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (BÀI TẬP 2)

Thực hiện đúng quy định liêm chính học thuật của học phần Chuyên đề tốt nghiệp 1, sinh viên khai báo minh bạch việc sử dụng công cụ AI hỗ trợ trong quá trình thực hiện Bài tập 2:

Công cụ AI	Dùng vào việc gì cụ thể	Áp dụng ở tệp / hàm nào	Em đã kiểm chứng thế nào?
Claude / ChatGPT / Antigravity	Gợi ý cấu trúc giải thuật nạp Idempotent trong DuckDB theo bộ ba khóa tự nhiên.	`src/load/loader.py`, hàm `load_fact_sales`	Đọc tài liệu DuckDB về ACID; viết test case `test_case_4_idempotent_double_run`; chạy pipeline 2 lần kiểm tra `COUNT(*)`.
Claude / ChatGPT / Antigravity	Gợi ý Regex và thứ tự ưu tiên parse 3 định dạng ngày tháng thực tế.	`src/quality/checker.py`, hàm `parse_date_flexibly`	Viết unit test cho các định dạng ngày biên; chạy trên tập 2.058 đơn hàng để kiểm tra tính chính xác và tỷ lệ thành công.
Claude / ChatGPT / Antigravity	Gợi ý mẫu layout 3 trạng thái (Loading, Empty, Error) và cấu hình theme Plotly cho Streamlit.	`app.py`, khối `st.spinner`, kiểm tra `df_filtered.empty` và `try...except` kết nối	Tự tạo 2 checkbox toggle trên Sidebar để kiểm tra trực quan giao diện ứng xử đúng theo rubric chấm điểm; đảm bảo chuỗi kết nối tuyệt đối không bị lộ.
— Không dùng — (Tự làm hoàn toàn)	Thiết kế kiến trúc phân lớp; xây dựng 6 hàm đo lường chất lượng dữ liệu; thiết kế lược đồ Star Schema; lập luận phân tích câu hỏi CH1, CH2, CH4; cơ chế phân quyền đăng nhập.	Toàn bộ cấu trúc dự án `src/`, file `sql/schema.sql`, kịch bản sinh dữ liệu `seeds/generate_seeds.py`	Tự thiết kế và đối chiếu trực tiếp với Hồ sơ BT1 đã nộp; tự viết toàn bộ câu truy vấn phân tích SQL sụt giảm 3 tháng liên tiếp (hàm cửa sổ `LAG`) và chạy kiểm thử pytest đạt 100% PASS.

LỜI CAM KẾT LIÊM CHÍNH HỌC THUẬT:

"*Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp trong báo cáo và kho mã nguồn này.*"

Xác nhận của Người khai báo:

Sinh viên đã cam kết điện tử.
Ngày hoàn tất: **05 tháng 11 năm 2026**

Sinh viên thực hiện

Mai Tuấn Kiệt
MSSV: **2374802010259**